



Security Audit

Wayfinder (PROMPT)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	14
Conclusion	76
Our Methodology	77
Disclaimers	79
About Hashlock	80

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Wayfinder team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Wayfinder AI is an AI-powered Web3 infrastructure platform focused on autonomous blockchain agents that can execute complex on-chain actions through natural language commands. The project enables users to interact with multiple blockchains using AI "agents" capable of handling swaps, bridging, lending, borrowing, smart contract deployment, and portfolio management without requiring deep technical knowledge. Its ecosystem is powered by the \$PROMPT token, which is used for governance, gas abstraction, staking, and premium platform functionality, positioning Wayfinder as an AI-driven omnichain automation layer for decentralized finance and blockchain operations.

Project Name: Wayfinder

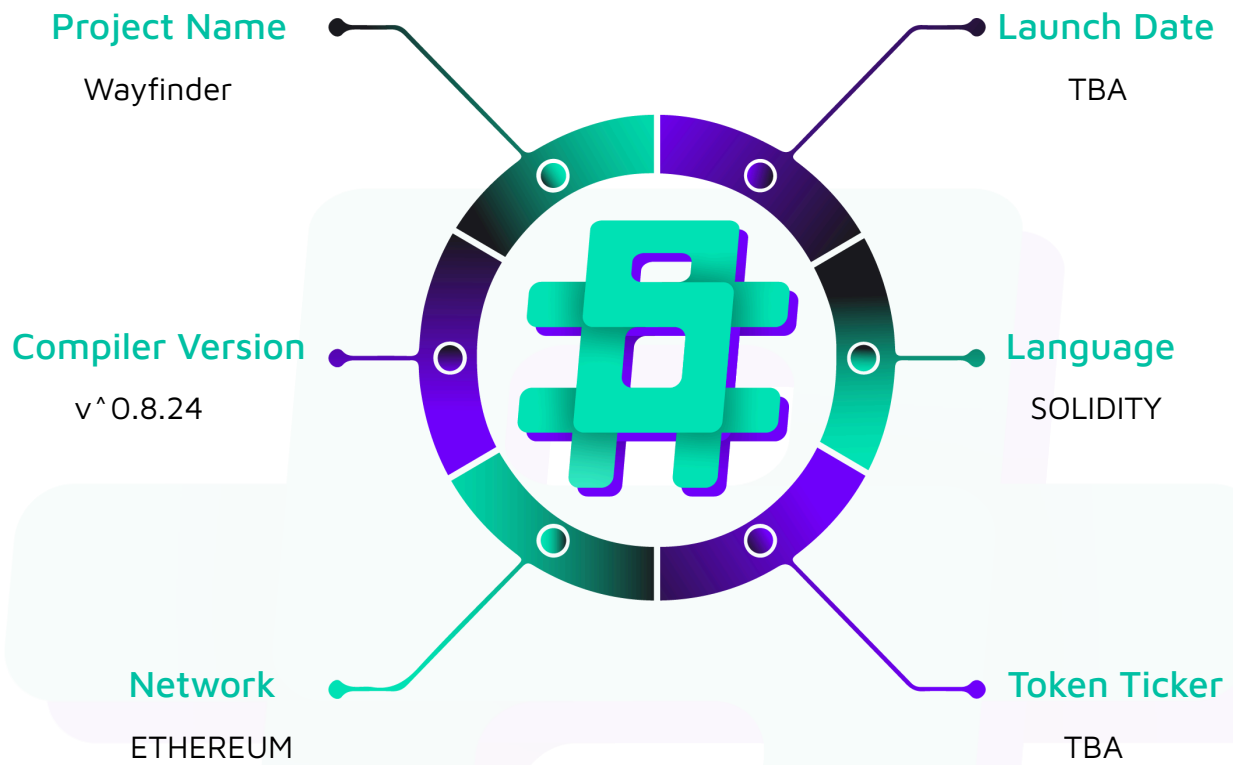
Project Type: PROMPT

Compiler Version: ^0.8.24

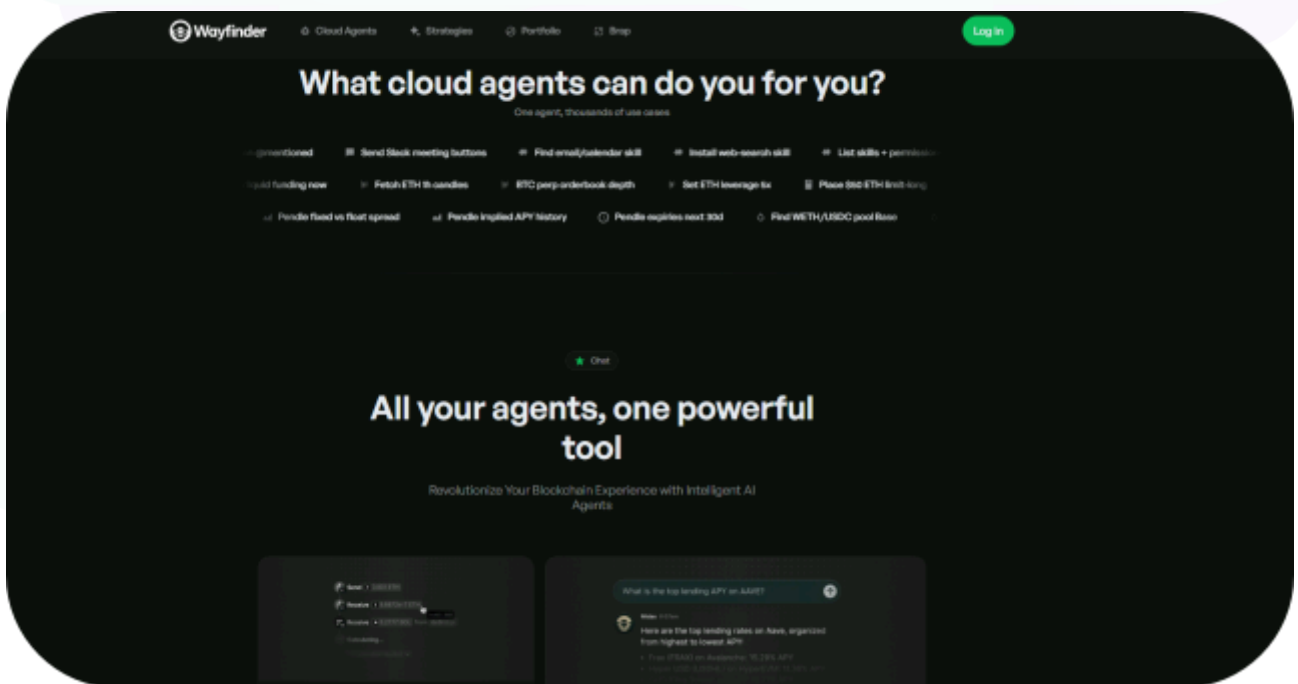
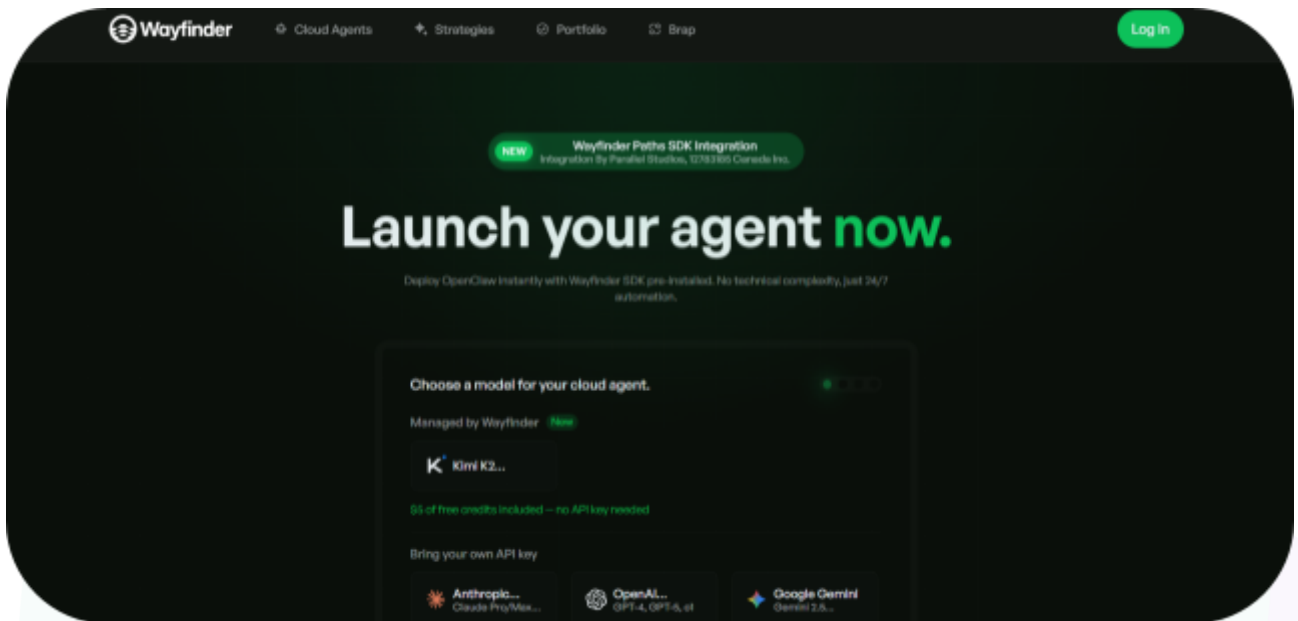
Website: <https://wayfinder.ai/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Wayfinder project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Wayfinder Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	May, 2026
Contract 1	PathBondManager.sol
Contract 2	MappingRewardsDistributor.sol
Audited GitHub Commit Hash	f7572cdc7f42e02d22830f42fc7b595ac9b01d52
Fix Review GitHub Commit Hash	70932a9b487d3d758cf1eff4335c29bd4ed0b8da

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

1 Medium severity vulnerability

14 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
PathBondManager.sol It allows admin and privileged roles to: • Set the safety fund. • Set the authorization signer. • Set the minimum challenge bond. • Set the minimum community stake. • Set timing parameters. • Set challenger reward basis points. • Set tier bond requirements. • Pause and unpause contract operations. • Grant and revoke roles. • Rescue surplus tokens. It allows path owners to: • Bond an initial path version with authorization. • Top up owner bond. • Announce a pending version upgrade with authorization. • Replace an initial pending version with authorization. • Promote a pending version after probation. • Expire a pending version. • Request path retirement. • Finalize path retirement. • Withdraw inactive owner bond. It allows community stakers to:	Contract achieves this functionality.

• Stake PROMPT into an active path. • Request unstake. • Withdraw stake after cooldown or exit window. It allows challengers to: • Open challenges against active, pending, historical, or superseded pending bundles. It allows resolver/admin to: • Resolve challenges as upheld or cleared. • Reject challenges as frivolous. It provides view functions for: • Path capital state. • Canonical path state. • Stake position. • Active and pending stake previews. • Bundle records. • Challenge records. • Open challenge ID. • Minimum bond values. • Bonded status. • Challengeable-until timestamps. • Path ID derivation.	
MappingRewardsDistributor.sol It allows admin to: • Update individual claimable balances. • Withdraw excess PROMPT tokens above total claimable liabilities. • Perform emergency withdrawal.	Contract achieves this functionality.

- Enable or disable claiming.
- Add or remove funders.

It allows funders to:

- Set claimable balances in batches.
- Prevent batch replay with batch IDs.

It allows users to:

- View claimable rewards by account and category.
- Claim creator rewards.
- Claim staker rewards.
- Claim multiple categories in one transaction.
- Claim all available rewards.

It enforces accounting checks for:

- Valid reward categories.
- Total claimable balances.
- Solvency before claims or re-enabling.
- Excess-token withdrawal limits.

Code Quality

This audit scope involves the smart contracts of the Wayfinder project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Wayfinder project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] PathBondManager#promotePendingVersion - Cleared/rejected pending challenge immunizes the later active bundle

Description

The protocol's challenge/slash mechanism is supposed to remain available across every lifecycle phase of a bundle (Pending -> Active -> Historical). When a bundle is challenged while it is still Pending, and that challenge is resolved as Cleared (resolver dismisses) or Rejected (frivolous), the per-record BundleResolution is permanently set to Cleared. If that same bundle is later promoted to Active, every subsequent challenge against it reverts. This effectively immunizes the bundle against slashing for the rest of its life, even though only one weak/sybil challenge was ever resolved against it.

Vulnerability Details

promotePendingVersion upgrades the bundle record from pending to active without resetting resolution:

```
PathTypes.BundleRecord storage pendingRecord =
    _bundleRecords[pathId][pendingBundleHash];
pendingRecord.wasActive = true;
pendingRecord.activatedAt = _toUint64(block.timestamp);
pendingRecord.challengeableUntil = 0;
```

src/PathBondManager.sol:683-686. Note pendingRecord.resolution is left untouched. If a previous pending-phase challenge was dismissed, resolution is Cleared:

```
_closeChallenge(
    challengeId,
    challenge,
    path,
    record,
```

```

    PathTypes.ChallengeStatus.Cleared,
    PathTypes.BundleResolution.Cleared
);

```

src/PathBondManager.sol:1352-1359 (and the symmetric `_resolveFrivolousChallenge` at src/PathBondManager.sol:1372-1379).

Once promoted, every gating check inside `_challengeTarget` rejects the bundle because the per-record `resolution` is non-None:

```

if (path.activeBundleHash == bundleHash && bundleHash != bytes32(0)) {
    PathTypes.BundleRecord storage activeRecord = _bundleRecords[pathId][bundleHash];
    if (
        activeRecord.resolution != PathTypes.BundleResolution.None
        || activeRecord.challengeOpen
        || (path.retirementReleaseTime != 0
            && path.retirementReleaseTime <= block.timestamp)
    ) {
        revert BundleNotChallengeable(pathId, bundleHash);
    }
    return PathTypes.ChallengeTarget.Active;
}

```

src/PathBondManager.sol:1457-1468. The same `resolution != None` short-circuit appears in the `Pending`, `SupersededPending`, and `Historical` arms (src/PathBondManager.sol:1446-1490). The failing invariant is `"a bundle that has never been the subject of an upheld slash on its current target must remain challengeable for that target's full window"`; it is broken because `BundleResolution` is treated as a single per-record terminal flag, ignoring which lifecycle phase it was set in.

Proof of Concept

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
import { PathBondManager } from "../../src/PathBondManager.sol";
import { PathTypes } from "../../src/libraries/PathTypes.sol";
import { MockERC20 } from "../mocks/MockERC20.sol";
import { Test } from "../helpers/Test.sol";

```

```

contract H01Actor {
    MockERC20 internal immutable token;
    PathBondManager internal immutable manager;
    constructor(MockERC20 t, PathBondManager m) {
        token = t;
        manager = m;
    }
    function approveManager(uint256 amount) external {
        token.approve(address(manager), amount);
    }
    function bondInitialAuthorized(
        bytes32 pathId,
        bytes32 bundleHash,
        PathTypes.RiskTier riskTier,
        uint256 amount,
        address owner,
        uint256 minAmount,
        uint256 nonce,
        uint64 deadline,
        bytes calldata sig
    ) external {
        manager.bondInitialAuthorized(pathId, bundleHash, riskTier, amount, owner,
minAmount, nonce, deadline, sig);
    }
    function announcePendingVersionAuthorized(
        bytes32 pathId,
        bytes32 bundleHash,
        uint256 extraBond,
        uint256 minAmount,
        uint256 nonce,
        uint64 deadline,
        bytes calldata sig
    ) external {
        manager.announcePendingVersionAuthorized(pathId, bundleHash, extraBond,
minAmount, nonce, deadline, sig);
    }
    function promotePendingVersion(bytes32 pathId) external {
        manager.promotePendingVersion(pathId);
    }
}

```

```

    }

    function openChallenge(bytes32 pathId, bytes32 bundleHash, bytes32 evidence)
external returns (uint256) {
    return manager.openChallenge(pathId, bundleHash, evidence);
}
}

contract H01ClearedResolutionImmunityTest is Test {
    uint256 internal constant TOKEN = 1e18;
    uint256 internal constant MIN_BOND = 100 * TOKEN;
    uint256 internal constant CHALLENGE_BOND = 10 * TOKEN;
    uint64 internal constant PROBATION_PERIOD = 7 days;
    uint64 internal constant CHALLENGE_WINDOW = 14 days;
    bytes32 internal constant EIP712_DOMAIN_TYPEHASH = keccak256(
        "EIP712Domain(string name,string version,uint256 chainId,address
verifyingContract)"
    );
    bytes32 internal constant AUTHORIZATION_TYPEHASH = keccak256(
        "PathAuthorization(bytes32 pathId,bytes32 bundleHash,address owner,uint8
riskTier,uint8 operation,uint256 minAmount,uint256 nonce,uint64 deadline)"
    );
    uint8 internal constant AUTH_OPERATION_BOND_INITIAL = 1;
    uint8 internal constant AUTH_OPERATION_ANNOUNCE_VERSION = 2;
    MockERC20 internal promptToken;
    PathBondManager internal manager;
    H01Actor internal ownerActor;
    H01Actor internal sybilActor;
    H01Actor internal challengerActor;
    uint256 internal authorizerPk = 0xA11CE;
    address internal authorizer;
    address internal safetyFund = address(0xBEEF);
    address internal admin = address(0xA11);
    address internal resolver = address(0xB22);
    address internal pauser = address(0xC33);
    bytes32 internal constant PATH_ID = keccak256("h01-path");
    bytes32 internal constant BUNDLE_PENDING = keccak256("h01-bundle");
    bytes32 internal constant EVIDENCE_HASH = keccak256("h01-evidence");
    function setUp() public {
        authorizer = vm.addr(authorizerPk);
    }
}

```

```

promptToken = new MockERC20("PROMPT", "PROMPT");
manager = new PathBondManager(
    PathBondManager.RoleConfig({ admin: admin, resolver: resolver, pauser:
pauser })),
    PathBondManager.ManagerConfig({
        promptToken: address(promptToken),
        safetyFund: safetyFund,
        authorizationSigner: authorizer,
        minChallengeBond: CHALLENGE_BOND,
        minCommunityStake: TOKEN,
        probationPeriod: PROBATION_PERIOD,
        challengeWindow: CHALLENGE_WINDOW,
        communityUnstakeCooldown: 1 days,
        pendingPenaltyPeriod: 7 days,
        challengerRewardBps: 2000,
        minOwnerBondByTier: [MIN_BOND, 500 * TOKEN, 1000 * TOKEN],
        minPendingBondByTier: [MIN_BOND, 500 * TOKEN, 1000 * TOKEN]
    })
);
ownerActor = new H01Actor(promptToken, manager);
sybilActor = new H01Actor(promptToken, manager);
challengerActor = new H01Actor(promptToken, manager);
promptToken.mint(address(ownerActor), 5000 * TOKEN);
promptToken.mint(address(sybilActor), 5000 * TOKEN);
promptToken.mint(address(challengerActor), 5000 * TOKEN);
ownerActor.approveManager(type(uint256).max);
sybilActor.approveManager(type(uint256).max);
challengerActor.approveManager(type(uint256).max);
}

function test_H01_clearedPendingChallenge_immunizesActiveBundle() public {
    // STEP 1: Owner bonds bundle as Pending.
    _bondInitial(BUNDLE_PENDING, PathTypes.RiskTier.ReadOnly, MIN_BOND);
    // STEP 2: A sybil opens a (weak / collusive) Pending challenge.
    uint256 cid = sybilActor.openChallenge(PATH_ID, BUNDLE_PENDING,
EVIDENCE_HASH);
    // STEP 3: Resolver clears it (BundleResolution.Cleared written to record).
    vm.prank(resolver);
    manager.resolveChallenge(cid, false, 0);
}

```



```

// STEP 4: Probation elapses, owner promotes the same bundle to Active.
vm.warp(block.timestamp + PROBATION_PERIOD + 1);
ownerActor.promotePendingVersion(PATH_ID);
// STEP 5: An honest challenger now sees genuinely malicious behavior in
// the active bundle and tries to slash it.
vm.expectRevert(
    abi.encodeWithSignature("BundleNotChallengeable(bytes32,bytes32)",
PATH_ID, BUNDLE_PENDING)
);
challengerActor.openChallenge(PATH_ID, BUNDLE_PENDING, EVIDENCE_HASH);
// The bundle is permanently immune for its entire active and historical
// lifecycle: ownerBondActive and communityStakeAssets can never be slashed.
}
// ===== Helpers =====
function _bondInitial(bytes32 bundleHash, PathTypes.RiskTier riskTier, uint256
amount) internal {
    uint256 nonce = manager.authorizationNonce(address(ownerActor),
AUTH_OPERATION_BOND_INITIAL);
    uint64 deadline = uint64(block.timestamp + 1 days);
    bytes memory sig = _signAuth(
        PATH_ID, bundleHash, address(ownerActor), riskTier,
AUTH_OPERATION_BOND_INITIAL,
        amount, nonce, deadline
    );
    ownerActor.bondInitialAuthorized(
        PATH_ID, bundleHash, riskTier, amount, address(ownerActor), amount,
nonce, deadline, sig
    );
}
function _domainSeparator() internal view returns (bytes32) {
    return keccak256(abi.encode(
        EIP712_DOMAIN_TYPEHASH,
        keccak256(bytes("WayfinderPathBondManager")),
        keccak256(bytes("2")),
        block.chainid,
        address(manager)
    ));
}

```

```

function _signAuth(
    bytes32 pathId,
    bytes32 bundleHash,
    address owner,
    PathTypes.RiskTier riskTier,
    uint8 operation,
    uint256 minAmount,
    uint256 nonce,
    uint64 deadline
) internal returns (bytes memory) {
    bytes32 structHash = keccak256(abi.encode(
        AUTHORIZATION_TYPEHASH, pathId, bundleHash, owner, riskTier, operation,
        minAmount, nonce, deadline
    ));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", _domainSeparator(),
        structHash));
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(authorizerPk, digest);
    return abi.encodePacked(r, s, v);
}
}

```

Impact

A path owner (or any sybil) can permanently disable slashing for a bundle by self-challenging it with a minimal `challengerBond` while it is `Pending`, having that challenge dismissed (`Cleared`) or rejected as frivolous, then promoting the bundle. Once active, no challenger can slash `ownerBondActive` or the community stake against that bundle — `openChallenge` reverts with `BundleNotChallengeable`. The `safetyFund` is therefore deprived of every slash it was meant to capture from a malicious active bundle, and stakers' protection collapses: the operator can produce arbitrarily harmful inferences with no economic recourse. The same `cleared` flag also kills challenges against the `Historical` phase after the bundle is later deactivated.

Recommendation

Reset the per-record resolution when the bundle moves into a new lifecycle phase, so a dismissed pending-phase challenge does not carry over to the active or historical phase. Apply this in `promotePendingVersion`:

```
PathTypes.BundleRecord storage pendingRecord =
  _bundleRecords[pathId][pendingBundleHash];
pendingRecord.wasActive = true;
pendingRecord.activatedAt = _toUint64(block.timestamp);
pendingRecord.challengeableUntil = 0;
+// A dismissed pending-phase challenge must not immunize the bundle in
+// its later active/historical phase. Reset the per-record verdict; it
+// is only meaningful while the record is in the phase that produced it.
+pendingRecord.resolution = PathTypes.BundleResolution.None;
```

Alternatively, scope `BundleResolution` per-target instead of per-record by indexing it as `mapping(ChallengeTarget => BundleResolution)` on `BundleRecord`, and have `_challengeTarget` only consult the entry for the lifecycle phase being challenged. Either fix restores the invariant that challengers can always slash an active bundle that has not yet been upheld-slashed in its current phase.

Status

Resolved

[H-02] PathBondManager#requestUnstake - Dust pre-staged unstake anchors stale cooldown and enables immediate full exit

Description

The protocol enforces a `communityUnstakeCooldown` between `requestUnstake` and `withdrawStake` so that stakers cannot exit instantly after a malicious update or other risk event becomes visible. The intent is *"every unstake request waits at least `communityUnstakeCooldown` from the moment it was requested"*. In the current implementation `cooldownEndsAt` is only written when there were zero pending shares before the call. By front-loading a 1-wei unstake request and waiting for that initial cooldown to pass, a staker leaves a stale, already-expired `cooldownEndsAt` on their position; any later `requestUnstake` (even for the entire stake) is treated as an "additional" request and reuses the old timestamp, so the staker can withdraw the full position in the same block.

Vulnerability Details

`requestUnstake` only sets `cooldownEndsAt` when `pendingSharesBefore == 0`:

```
uint256 pendingSharesBefore = position.pendingUnstakeShares;
position.activeShares -= shares;
position.pendingUnstakeShares = pendingSharesBefore + shares;
if (pendingSharesBefore == 0) {
    position.cooldownEndsAt = _toUint64(block.timestamp + communityUnstakeCooldown);
}
```

`src/PathBondManager.sol:757-762`. Once a 1-wei dust unstake is staged, `pendingUnstakeShares` is non-zero forever after, so every subsequent call leaves `cooldownEndsAt` untouched.

`withdrawStake` consumes that stale value as the unlock gate for the full pending balance:

```
uint256 shares = position.pendingUnstakeShares;
uint256 activeSharesToWithdraw = 0;
uint64 unlockTime = position.cooldownEndsAt;
if (path.communityExitUnlockTime != 0) {
```

```

    shares += position.activeShares;
    activeSharesToWithdraw = position.activeShares;
    unlockTime = path.communityExitUnlockTime;
    if (
        position.cooldownEndsAt != 0 && position.pendingUnstakeShares != 0
        && position.cooldownEndsAt < unlockTime
    ) {
        unlockTime = position.cooldownEndsAt;
    }
    ...
    if (unlockTime == 0 || block.timestamp < unlockTime) {
        revert WithdrawalLocked(unlockTime);
    }

```

src/PathBondManager.sol:793-813. Because `cooldownEndsAt` is in the past, the gate passes immediately. Worse, in the deactivation branch the inner if actively prefers the stale `cooldownEndsAt` over `path.communityExitUnlockTime`, so the same anchor also bypasses the post-deactivation challenge-window exit. The failing invariant is `"between requesting the dominant share of a position and withdrawing it, at least communityUnstakeCooldown must elapse"`.

Proof of Concept

The existing test `test/audit/Triage_B1_B2_StakerCooldownBypass.t.sol::test_B1_dustUnstake_anchorsCooldown_letsFullStakeExitImmediately` demonstrates the exploit end-to-end. Reproduced here for clarity:

```

/// @notice H-02: dust pre-staged unstake anchors stale cooldown and lets the
/// staker withdraw their full position in the same block they request it.
function test_H02_dustUnstake_anchorsCooldown_letsFullStakeExitImmediately() public {
    // STEP 1: Bond, promote, stake the full 100e18.
    _bondInitial(BUNDLE_ONE, PathTypes.RiskTier.ReadOnly, 200 * TOKEN);
    vm.warp(block.timestamp + PROBATION_PERIOD + 1);
    ownerActor.promotePendingVersion(PATH_ID);
    uint256 stakeAmount = 100 * TOKEN;
    stakerActor.stake(PATH_ID, stakeAmount);

```

```

uint256 stakerBalanceAfterStake = stakerActor.promptBalance();
// STEP 2: Stage a 1-wei pending unstake at t0. cooldownEndsAt is set
// exactly once because pendingUnstakeShares was previously 0.
stakerActor.requestUnstake(PATH_ID, 1);
PathTypes.StakePosition memory pos1 = manager.getStakePosition(PATH_ID,
address(stakerActor));
assertEq(pos1.pendingUnstakeShares, 1, "pending = 1");
assertGt(pos1.cooldownEndsAt, block.timestamp, "cooldown set into future");
uint64 anchoredCooldown = pos1.cooldownEndsAt;
// STEP 3: Wait far past the cooldown (months).
vm.warp(block.timestamp + 60 days);
// STEP 4: Now request the entire remaining position. Because
// pendingSharesBefore != 0, requestUnstake skips the cooldown write.
uint256 remainingAssets = stakeAmount - 1;
stakerActor.requestUnstake(PATH_ID, remainingAssets);
PathTypes.StakePosition memory pos2 = manager.getStakePosition(PATH_ID,
address(stakerActor));
assertEq(pos2.cooldownEndsAt, anchoredCooldown, "cooldown is NOT refreshed");
assertTrue(pos2.cooldownEndsAt < block.timestamp, "anchored cooldown is in the
past");
// STEP 5: withdrawStake succeeds in the same block: cooldownEndsAt is
// already in the past, so the unlock gate passes immediately.
stakerActor.withdrawStake(PATH_ID);
uint256 stakerBalanceAfterWithdraw = stakerActor.promptBalance();
// The attacker recovered effectively their full original deposit
// without waiting communityUnstakeCooldown after the *real* unstake
// request, violating the cooldown invariant.
assertTrue(
    stakerBalanceAfterWithdraw - stakerBalanceAfterStake >= stakeAmount - 1,
    "H-02 demonstrated: full stake withdrawn with zero effective cooldown"
);
}

```

Impact

Any staker can permanently neutralize the unstake cooldown for themselves with a single 1-wei `requestUnstake` at the moment they first stake. Months later, when negative information about the path appears (e.g. an imminent challenge, a quality

regression, governance action), they can call `requestUnstake` for the entire remainder and immediately call `withdrawStake` in the same block. This defeats the underwriting role community stake plays — it is the soft-loss buffer that absorbs slashing of an active bundle — because slashable capital can leave the system on demand. As demonstrated by the existing `test_B2_activeShareEarlyExit_via_anchoredCooldown_after_pendingSlash` companion case, the same anchor also bypasses the `communityExitUnlockTime` window after a deactivating slash, letting community stake escape the post-challenge exit lock.

Recommendation

Refresh `cooldownEndsAt` on every `requestUnstake` rather than only on the first. Use `max(existing, now + communityUnstakeCooldown)` so previously-pending shares never have their unlock time shortened, while newly-pending shares always observe a full cooldown:

```
uint256 pendingSharesBefore = position.pendingUnstakeShares;
position.activeShares -= shares;
position.pendingUnstakeShares = pendingSharesBefore + shares;
-if (pendingSharesBefore == 0) {
-    position.cooldownEndsAt = _toUint64(block.timestamp + communityUnstakeCooldown);
-}
+uint64 newCooldown = _toUint64(block.timestamp + communityUnstakeCooldown);
+if (newCooldown > position.cooldownEndsAt) {
+    position.cooldownEndsAt = newCooldown;
+}
```

This restores the invariant that the largest pending-unstake request always observes at least `communityUnstakeCooldown`, eliminates the dust-anchor exploit, and as a side effect closes the companion bypass of `communityExitUnlockTime` after a deactivating slash because the inner branch in `withdrawStake` can no longer pick up an already-elapsed `cooldownEndsAt`.

Status

Resolved

Medium

[M-01] PathBondManager#withdrawStake - Active shares can exit before communityExitUnlockTime via stale pending cooldown

Description

When an active bundle is deactivated via a slash that drops ownerBondActive below the active-tier minimum, `_deactivateActiveBundle` calls `_startOrPreserveCommunityExit` to set `path.communityExitUnlockTime` — a forward-dated quiet window during which community shares (active + pending) must remain locked so that follow-on Historical/SupersededPending challenges and off-chain reward accounting can settle. However, `withdrawStake` contains a stale-cooldown override that lets a staker holding any non-zero pending-unstake shares with an already-expired personal `cooldownEndsAt` pull the effective `unlockTime` back to that past timestamp, releasing **both** the freshly-included active shares and the pending shares immediately. The community-exit gate is silently bypassed.

Vulnerability Details

The relevant branch in `withdrawStake` is at `src/PathBondManager.sol:793-809`:

```
uint256 shares = position.pendingUnstakeShares;
uint256 activeSharesToWithdraw = 0;
uint64 unlockTime = position.cooldownEndsAt;
if (path.communityExitUnlockTime != 0) {
    shares += position.activeShares;
    activeSharesToWithdraw = position.activeShares;
    unlockTime = path.communityExitUnlockTime;
    if (
        position.cooldownEndsAt != 0 && position.pendingUnstakeShares != 0
        && position.cooldownEndsAt < unlockTime
    ) {
        unlockTime = position.cooldownEndsAt;
    }
}
if (path.retirementReleaseTime != 0 && path.retirementReleaseTime > unlockTime) {
```



```

    unlockTime = path.retirementReleaseTime;
}

```

When `path.communityExitUnlockTime != 0`, `shares` is mutated to include `position.activeShares` and `activeSharesToWithdraw` is set to release them. `unlockTime` is then set to the community-exit deadline (correct). The override at `src/PathBondManager.sol:800-805` then checks "do you have any pending shares with a cooldown earlier than the exit window?" — and if yes, lowers `unlockTime` back to that personal cooldown. Critically, the override applies to the **combined** shares total (active + pending) rather than to the pending-shares portion only, so the active shares — which were never subject to a personal cooldown and which the community-exit window was specifically designed to gate — are released alongside the pending dust. This is a structural mismatch with the retirement guard immediately below at `src/PathBondManager.sol:807-809`, which only **raises** `unlockTime` when `path.retirementReleaseTime` is non-zero. Deactivation-driven exits clear `retirementReleaseTime` (`_deactivateActiveBundle` at `src/PathBondManager.sol:1868: path.retirementReleaseTime = 0;`), so there is no symmetric upper bound that would re-anchor active shares to `communityExitUnlockTime` after the line 800-805 override has lowered it.

Proof of Concept

The following Foundry test (adapted from `test/audit/Triage_B1_B2_StakerCooldownBypass.t.sol::test_B2_activeShareEarlyExit_via_anchoredCooldown_after_pendingSlash`) demonstrates the bypass end to end. The staker pre-stages a 1-wei pending-unstake anchor, waits past their personal cooldown, then triggers a deactivation via a Pending-bundle slash large enough to spill into the active bond. `withdrawStake` succeeds in the same block, even though `communityExitUnlockTime` is `block.timestamp + CHALLENGE_WINDOW` in the future:

```

function test_M01_activeShareEarlyExit_via_anchoredCooldown_after_pendingSlash()
public {
    // Bond active bundle exactly at the active-tier minimum so any active
    // slash forces _deactivateActiveBundle.
    _bondInitial(BUNDLE_ONE, PathTypes.RiskTier.ReadOnly, MIN_BOND);
}

```

```

vm.warp(block.timestamp + PROBATION_PERIOD + 1);
ownerActor.promotePendingVersion(PATH_ID);
uint256 stakeAmount = 1000 * TOKEN;
stakerActor.stake(PATH_ID, stakeAmount);
uint256 stakerBalanceAfterStake = stakerActor.promptBalance();
// 1-wei pending unstake anchors cooldownEndsAt (the H-02 / B-1 prerequisite).
stakerActor.requestUnstake(PATH_ID, 1);
// Personal cooldown elapses; community-exit window has NOT started yet.
vm.warp(block.timestamp + COMMUNITY_UNSTAKE_COOLDOWN + 1);
// Owner announces a pending upgrade so a Pending challenge can spill.
_announcePending(BUNDLE_TWO, MIN_BOND);
// Resolver upholds with slash big enough to spill from ownerBondPending
// into ownerBondActive, deactivating the active bundle and arming
// communityExitUnlockTime.
uint256 cid = challengerActor.openChallenge(PATH_ID, BUNDLE_TWO, EVIDENCE_HASH);
vm.prank(resolver);
manager.resolveChallenge(cid, true, 2 * MIN_BOND);
PathTypes.PathCapital memory afterSlash = manager.getPathCapital(PATH_ID);
assertEq(afterSlash.activeBundleHash, bytes32(0), "active deactivated");
assertEq(afterSlash.retirementReleaseTime, 0, "deactivation clears retirement
timer");
assertGt(afterSlash.communityExitUnlockTime, block.timestamp, "exit window in
future");
// BUG: cooldownEndsAt (in the past) overrides communityExitUnlockTime
// for BOTH active and pending shares.
stakerActor.withdrawStake(PATH_ID);
uint256 stakerBalanceAfterWithdraw = stakerActor.promptBalance();
assertGe(
    stakerBalanceAfterWithdraw - stakerBalanceAfterStake,
    stakeAmount - 1,
    "M-01: active+pending shares exited before communityExitUnlockTime"
);
}

```

Impact

The community-exit window is the protocol's enforced quiet period between active-bundle deactivation and full unwind — the time during which `Historical` and `SupersededPending` challenges can still be raised against the slashed bundle and during



which off-chain reward accounting reconciles the path's end-of-life balances. Letting active shares exit at an expired personal cooldown erases that window's integrity: stakers race out, the community pool drains, and any post-deactivation slash has no shares left to enforce against. Combined with H-02 — which lets a staker mint a 1-wei pending anchor for negligible cost — this becomes an unconditional, cheap, and repeatable bypass: any attentive staker can pre-stage a stale cooldown anchor and then exit instantly the moment any deactivation event sets `communityExitUnlockTime`.

Recommendation

The early-cooldown override must apply only to the ****pending-shares portion**** of the withdrawal, never to the active-shares portion that the community-exit window is designed to gate. The simplest fix is to defer including active shares when the personal cooldown is older than `communityExitUnlockTime`, and require a second `withdrawStake` call once `communityExitUnlockTime` is reached. Diff against `src/PathBondManager.sol:793-809`:

```
uint256 shares = position.pendingUnstakeShares;
uint256 activeSharesToWithdraw = 0;
uint64 unlockTime = position.cooldownEndsAt;
if (path.communityExitUnlockTime != 0) {
-   shares += position.activeShares;
-   activeSharesToWithdraw = position.activeShares;
-   unlockTime = path.communityExitUnlockTime;
-   if (
-       position.cooldownEndsAt != 0 && position.pendingUnstakeShares != 0
-       && position.cooldownEndsAt < unlockTime
-   ) {
-       unlockTime = position.cooldownEndsAt;
-   }
+   // Active shares are gated by the community-exit window; the
+   // pending-side personal cooldown must NEVER lower their release time.
+   if (
+       position.pendingUnstakeShares != 0 && position.cooldownEndsAt != 0
+       && position.cooldownEndsAt < path.communityExitUnlockTime
+   ) {
+       // Release pending-only at the (already-elapsed) personal cooldown;
```

```

+      // active shares wait for communityExitUnlockTime in a later call.
+      unlockTime = position.cooldownEndsAt;
+    } else {
+      shares += position.activeShares;
+      activeSharesToWithdraw = position.activeShares;
+      unlockTime = path.communityExitUnlockTime;
+    }
+  }
+  if (path.retirementReleaseTime != 0 && path.retirementReleaseTime > unlockTime) {
+    unlockTime = path.retirementReleaseTime;
+  }

```

Equivalently — and arguably safer as a one-line invariant — replace the override entirely with a hard floor: when `path.communityExitUnlockTime != 0` and `activeSharesToWithdraw != 0`, require `block.timestamp >= path.communityExitUnlockTime` regardless of `position.cooldownEndsAt`. This guarantees that no stale personal cooldown can ever release active shares before the community-exit deadline.

Status

Resolved

Low

[L-01] PathBondManager#requestUnstake - Dust-share staker whose asset value rounds to zero cannot voluntarily unstake

Description

After a large slash on an active bundle, a community staker can be left with a non-zero `activeShares` balance that rounds down to zero `activeAssets`. `requestUnstake` rejects `amount == 0` and rejects any positive amount above the rounded-down asset balance, leaving the position mechanically stuck outside the shared community-exit path.

Vulnerability Details

`requestUnstake` validates the requested amount against the staker's asset value computed by `_convertToAssets`, which rounds shares to assets via integer division.

```
function requestUnstake(
    bytes32 pathId,
    uint256 amount
) external whenNotPaused {
    if (amount == 0) revert InvalidAmount();
    ...
    uint256 activeAssets = _convertToAssets(
        position.activeShares, path.communityStakeAssets, path.totalCommunityShares
    );
    if (amount > activeAssets) revert InsufficientStake(amount, activeAssets);
}
```

`src/PathBondManager.sol:733, src/PathBondManager.sol:742-745.`

`_convertToAssets` rounds down:

```
function _convertToAssets(
    uint256 shares,
    uint256 totalAssets,
    uint256 totalShares
) private pure returns (uint256) {
    if (shares == 0 || totalAssets == 0 || totalShares == 0) return 0;
    if (shares == totalShares) return totalAssets;
}
```

```

    return (shares * totalAssets) / totalShares;
}

```

src/PathBondManager.sol:2096-2103.

If `position.activeShares * path.communityStakeAssets < path.totalCommunityShares`, `activeAssets` is 0. `amount == 0` reverts `InvalidAmount()`; any positive `amount` reverts `InsufficientStake`. The staker cannot exit individually until a community-exit unlock is triggered by some other lifecycle event.

Proof of Concept

```

function test_DustStakerCannotUnstake() public {
    bytes32 pathId = _seedActivePath();
    address staker = makeAddr("staker");
    // staker deposits a tiny amount; total pool grows large
    _stake(pathId, staker, 1);
    _stake(pathId, makeAddr("whale"), 1_000_000e18);
    // dispute upheld; community slashed nearly all assets
    _slashCommunityToDust(pathId);
    // staker still has activeShares > 0, but rounds to 0 assets
    (uint256 activeAssets,) = manager.previewStakeAssets(pathId, staker);
    assertEq(activeAssets, 0);
    vm.startPrank(staker);
    vm.expectRevert(PathBondManager.InvalidAmount.selector);
    manager.requestUnstake(pathId, 0);
    vm.expectRevert(); // InsufficientStake(1, 0)
    manager.requestUnstake(pathId, 1);
    vm.stopPrank();
}

```

Impact

A small staker whose pro-rata share of `communityStakeAssets` rounds to zero (e.g., after a maximum-magnitude active slash that removes most assets while leaving total shares unchanged) loses the ability to use `requestUnstake`. Their dust shares remain trapped in the active pool until the path enters community exit through another route, locking

grievable dust positions and producing accounting noise on `totalActiveCommunityShares`.

Recommendation

Allow a dust staker to burn all `activeShares` for whatever they round to (zero) by accepting an explicit "exit-all" path:

```
function requestUnstake(
    bytes32 pathId,
    uint256 amount
) external whenNotPaused {
-   if (amount == 0) revert InvalidAmount();
+   PathTypes.PathCapital storage path = _requireExistingPath(pathId);
+   _requireNoOpenChallenges(pathId);
+   if (path.communityExitUnlockTime != 0) {
+       revert CommunityExitInProgress(pathId, path.communityExitUnlockTime);
+   }
+   PathTypes.StakePosition storage position = _stakePosition(pathId, msg.sender);
+   uint256 activeAssets = _convertToAssets(
+       position.activeShares, path.communityStakeAssets, path.totalCommunityShares
+   );
+   if (amount == 0) {
+       if (position.activeShares == 0) revert InvalidAmount();
+       // burn all dust shares; assets is 0 by definition
+       uint256 dustShares = position.activeShares;
+       position.activeShares = 0;
+       path.totalActiveCommunityShares -= dustShares;
+       emit DustStakeBurned(pathId, msg.sender, dustShares);
+       return;
+   }
-
-   PathTypes.PathCapital storage path = _requireExistingPath(pathId);
-   ...
```

Status

Resolved

[L-02] PathBondManager#withdrawStake - Stale active-only position cleanup rolls back

Description

`withdrawStake` zeroes a stake position whose `stakeEpoch` is stale, but only returns early when there were stale pending shares to invalidate. A stale position that holds only `activeShares` (no pending) falls through to the `NoStakeToWithdraw` revert, which reverts the whole transaction and rolls back the cleanup writes, leaving the stale position permanently uncleaned via this entrypoint.

Vulnerability Details

```
_ensureStakeEpoch(path);
PathTypes.StakePosition storage position = _stakePositions[pathId][msg.sender];
if (position.stakeEpoch != path.stakeEpoch) {
    uint64 staleEpoch = position.stakeEpoch;
    uint256 stalePendingShares = position.pendingUnstakeShares;
    position.activeShares = 0;
    position.pendingUnstakeShares = 0;
    position.cooldownEndsAt = 0;
    position.stakeEpoch = path.stakeEpoch;
    if (stalePendingShares != 0) {
        emit PendingUnstakeInvalidated(pathId, msg.sender, staleEpoch,
path.stakeEpoch);
        return;
    }
}
...
if (shares == 0) revert NoStakeToWithdraw(pathId, msg.sender);
```

`src/PathBondManager.sol:776-791, src/PathBondManager.sol:810.`

When the stale position has only active shares (`stalePendingShares == 0`), the function does not return. Instead it computes `shares = position.pendingUnstakeShares = 0` and reverts `NoStakeToWithdraw`. The revert undoes the `position.activeShares = 0` and the `position.stakeEpoch = path.stakeEpoch` updates.

Proof of Concept

```
function test_StaleActiveOnlyPositionRollsBack() public {
    bytes32 pathId = _seedActivePath();
    address staker = makeAddr("staker");
    _stake(pathId, staker, 100e18);
    // staker never requests unstake; only active shares present
    _bumpStakeEpoch(pathId); // e.g. via permanent slash or full community exit
    // canonical position is stale; no pending shares
    PathTypes.StakePosition memory pre = manager.getStakePosition(pathId, staker);
    vm.prank(staker);
    vm.expectRevert(
        abi.encodeWithSelector(PathBondManager.NoStakeToWithdraw.selector, pathId,
    staker)
    );
    manager.withdrawStake(pathId);
    // raw storage still carries stale epoch + activeShares because revert rolled
    back
    PathTypes.StakePosition memory post = _readRawPosition(pathId, staker);
    assertEq(post.activeShares, pre.activeShares);
}
```

Impact

Stakers caught at the boundary of an epoch reset where they held only active shares (and no pending) cannot use `withdrawStake` as a no-op cleanup; the call always reverts. They cannot reset their stale-epoch state through this entrypoint, and the canonical position remains visibly stale until they happen to call a path that doesn't depend on `withdrawStake`. Mostly informational noise, but users following the documented sequence will see persistent reverts.

Recommendation

Emit and `return` even when only active shares were stale, so the cleanup persists:

```
-         if (stalePendingShares != 0) {
-             emit PendingUnstakeInvalidated(pathId, msg.sender, staleEpoch,
path.stakeEpoch);
```

```
-         return;
-     }
+     if (stalePendingShares != 0) {
+         emit PendingUnstakeInvalidated(pathId, msg.sender, staleEpoch,
path.stakeEpoch);
+     }
+     // Always exit after epoch reset so the cleanup writes persist; the
+     // staker has nothing left to withdraw at this epoch.
+     return;
```

Status

Resolved

[L-03] PathBondManager#bondInitialAuthorized - Lacks the open-challenge freeze guard

Description

The README states an open challenge freezes the entire path lifecycle, and every other lifecycle entrypoint enforces this with `_requireNoOpenChallenges(pathId)`. `bondInitialAuthorized` is the lone outlier and never calls the helper, so a path owner can announce a fresh initial pending bundle on a path while a challenge is still open against that path's state.

Vulnerability Details

`bondInitialAuthorized` performs validation but never invokes `_requireNoOpenChallenges`:

```
function bondInitialAuthorized(
    bytes32 pathId,
    bytes32 bundleHash,
    PathTypes.RiskTier riskTier,
    uint256 amount,
    address owner,
    uint256 minAmount,
    uint256 nonce,
    uint64 deadline,
    bytes calldata signature
) external whenNotPaused nonReentrant {
    if (pathId == bytes32(0) || bundleHash == bytes32(0)) revert InvalidIdentifier();
    _validateRiskTier(riskTier);
    _consumeAuthorization(...);
    PathTypes.PathCapital storage path = _paths[pathId];
    _ensurePathOwner(pathId, path, owner);
    _reconcileSupersededPendingReserve(pathId, path);
    _reconcileInactiveOwnerBond(pathId, path);
    if (path.permanentlySlashed) revert SlashedPath(pathId);
    if (block.timestamp < path.pendingPenaltyEndsAt) {
        revert PendingPenaltyActive(path.pendingPenaltyEndsAt);
    }
}
```

src/PathBondManager.sol:451-484.

Sibling functions all guard against open challenges, e.g. `topUpOwnerBond` at src/PathBondManager.sol:522, `announcePendingVersionAuthorized` at src/PathBondManager.sol:573, and `expirePendingVersion` at src/PathBondManager.sol:920.

Proof of Concept

```
function test_BondInitialBypassesChallengeFreeze() public {
    bytes32 pathId = _setupPathWithOpenSupersededChallenge();
    // path has openChallenge set, no active or pending bundle
    assertTrue(manager.hasOpenChallenge(pathId));
    assertEquals(manager.getPathCapital(pathId).activeBundleHash, bytes32(0));
    assertEquals(manager.getPathCapital(pathId).pendingBundleHash, bytes32(0));
    bytes32 newBundle = keccak256("v2");
    (bytes memory sig, uint256 nonce, uint64 deadline) =
        _signBondInitial(pathId, newBundle, RiskTier.Low, BOND, owner, BOND);
    vm.prank(owner);
    // succeeds despite challenge being open against the path
    manager.bondInitialAuthorized(
        pathId, newBundle, RiskTier.Low, BOND, owner, BOND, nonce, deadline, sig
    );
    // contrast: every other lifecycle call would have reverted ChallengeAlreadyOpen
}
```

Impact

If a path reaches a state where it has no `activeBundleHash` and no `pendingBundleHash` while a challenge is still open against a previously-tracked bundle (e.g. an outstanding `SupersededPending` challenge in an edge configuration), the owner can call `bondInitialAuthorized` to register a brand-new initial pending bundle. This bypasses the documented "all lifecycle frozen during challenge" invariant and confuses off-chain consumers tracking the dispute state.

Recommendation

```
function bondInitialAuthorized(
```

```
...
) external whenNotPaused nonReentrant {
    if (pathId == bytes32(0) || bundleHash == bytes32(0)) revert
InvalidIdentifier();
    _validateRiskTier(riskTier);
+   _requireNoOpenChallenges(pathId);
    _consumeAuthorization(
        ...
    );
}
```

Status

Resolved

[L-04] PathBondManager#expirePendingVersion - Superseded pending reserve and canonical state drift after pending expiry

Description

`expirePendingVersion` clears the pending bundle without first calling `_reconcileSupersededPendingReserve`, so a path that previously had its initial pending bundle replaced can keep `ownerBondSupersededPending` and `supersededPendingBundleHash` populated. `getCanonicalPathState` does not consult those fields, so it returns `Idle` for a path that still holds locked superseded reserve.

Vulnerability Details

`expirePendingVersion` only reconciles inactive owner bond:

```
function expirePendingVersion(
    bytes32 pathId
) external whenNotPaused {
    PathTypes.PathCapital storage path = _requirePathOwner(pathId);
    _reconcileInactiveOwnerBond(pathId, path);
    if (path.pendingBundleHash == bytes32(0)) revert NoPendingVersion(pathId);
    _requireNoOpenChallenges(pathId);
    if (block.timestamp < path.probationEndsAt) {
        revert ProbationNotFinished(path.probationEndsAt);
    }
    _expirePendingVersion(pathId, path);
}
```

src/PathBondManager.sol:914-926.

`getCanonicalPathState` ignores `supersededPendingBundleHash` / `ownerBondSupersededPending` when no active or pending bundle exists:

```
if (
    path.communityStakeAssets != 0 || path.ownerBondActive != 0
    || _historicalOwnerBondLocked(path)
) {
    return PathTypes.CanonicalPathState.DeactivatedAwaitingRelease;
}
if (path.pendingBundleHash != bytes32(0)) {
```

```

    return PathTypes.CanonicalPathState.InitialProbation;
}
return PathTypes.CanonicalPathState.Idle;

```

src/PathBondManager.sol:1053-1062.

The reconciliation helper that frees the superseded reserve into `ownerBondInactive` is `_reconcileSupersededPendingReserve` at `src/PathBondManager.sol:1789-1809`, but `expirePendingVersion` never calls it.

Proof of Concept

```

function test_SupersededReserveDriftAfterExpiry() public {
    bytes32 pathId = keccak256("path");
    bytes32 v1 = keccak256("v1");
    bytes32 v2 = keccak256("v2");
    _bondInitial(pathId, v1, BOND);
    _replaceInitialPending(pathId, v1, v2, BOND); // ownerBondSupersededPending =
BOND
    // wait out probation
    vm.warp(block.timestamp + manager.probationPeriod() + 1);
    vm.prank(owner);
    manager.expirePendingVersion(pathId);
    PathTypes.PathCapital memory p = manager.getPathCapital(pathId);
    assertGt(p.ownerBondSupersededPending, 0); // reserve still locked
    assertEq(uint256(manager.getCanonicalPathState(pathId)),
        uint256(PathTypes.CanonicalPathState.Idle)); // misleading
}

```

Impact

Off-chain consumers reading `getCanonicalPathState` see `Idle` for a path that still holds locked `ownerBondSupersededPending` (e.g., flow: `bond initial` → `replaceInitialPendingVersionAuthorized` → `wait probation` → `expirePendingVersion`). The funds are recoverable on the next call into `topUpOwnerBond/bondInitialAuthorized` because those entrypoints invoke `_reconcileSupersededPendingReserve`, but the canonical view is misleading and the reserve cannot be withdrawn until the owner triggers another reconciling entrypoint.

Recommendation

Reconcile the superseded reserve inside `expirePendingVersion` and surface it from the canonical view:

```
function expirePendingVersion(
    bytes32 pathId
) external whenNotPaused {
    PathTypes.PathCapital storage path = _requirePathOwner(pathId);
+   _reconcileSupersededPendingReserve(pathId, path);
    _reconcileInactiveOwnerBond(pathId, path);
    if (path.pendingBundleHash == bytes32(0)) revert NoPendingVersion(pathId);
```

And/or include the superseded reserve in the `DeactivatedAwaitingRelease` predicate so the canonical view reflects locked funds.

Status

Resolved

[L-05] PathBondManager#replaceInitialPendingVersionAuthorized - Expired initial pending bundle can be replaced, extending probation

Description

`replaceInitialPendingVersionAuthorized` neither auto-expires a matured initial pending bundle nor checks whether probation is still live, while its sibling `announcePendingVersionAuthorized` does call `_expirePendingVersionIfPossible` first. As a result, an owner holding any unused authorization signature can keep an initial pending probation alive indefinitely by replacing the same expired bundle, instead of letting the path expire normally.

Vulnerability Details

`announcePendingVersionAuthorized` defends against stale state:

```
PathTypes.PathCapital storage path = _requirePathOwner(pathId);
_reconcileSupersededPendingReserve(pathId, path);
_reconcileInactiveOwnerBond(pathId, path);
_expirePendingVersionIfPossible(pathId, path);
```

src/PathBondManager.sol:551-554.

`replaceInitialPendingVersionAuthorized` skips that step:

```
PathTypes.PathCapital storage path = _requirePathOwner(pathId);
_reconcileSupersededPendingReserve(pathId, path);
_reconcileInactiveOwnerBond(pathId, path);
if (path.permanentlySlashed) revert SlashedPath(pathId);
if (path.activeBundleHash != bytes32(0)) revert InitialProbationOnly(pathId);
if (path.pendingBundleHash == bytes32(0)) revert NoPendingVersion(pathId);
if (path.pendingBundleHash != previousBundleHash) {
    revert PendingVersionMismatch(pathId, path.pendingBundleHash,
previousBundleHash);
}
...
_announcePending(pathId, path, bundleHash);
```

src/PathBondManager.sol:607-651.

There is no `block.timestamp < path.probationEndsAt` check, and

`_expirePendingVersionIfPossible` is never invoked. `_announcePending` resets `probationEndsAt` to a fresh window.

Proof of Concept

```
function test_ExpiredPendingCanBeReplaced() public {
    bytes32 pathId = keccak256("path");
    bytes32 v1 = keccak256("v1");
    bytes32 v2 = keccak256("v2");
    _bondInitial(pathId, v1, BOND); // probation starts
    vm.warp(block.timestamp + manager.probationPeriod() + 1);
    // probationEndsAt is now in the past; expirePendingVersion would succeed
    (bytes memory sig, uint256 nonce, uint64 deadline) =
        _signReplace(pathId, v1, v2, BOND, owner, BOND);
    vm.prank(owner);
    manager.replaceInitialPendingVersionAuthorized(
        pathId, v1, v2, BOND, BOND, nonce, deadline, sig
    );
    PathTypes.PathCapital memory p = manager.getPathCapital(pathId);
    assertGt(p.probationEndsAt, block.timestamp); // probation extended
    assertEq(p.pendingBundleHash, v2);
}
```

Impact

A path that should have transitioned out of initial probation (probation matured but not yet expired) can be force-rolled into a brand-new probation window by replacing the matured pending bundle. This effectively lets owners with a pre-signed replacement authorization perpetually defer expiry, blocking community exit on `ownerBondPending` and keeping the path in a probationary state when it should have already cleared.

Recommendation

```
PathTypes.PathCapital storage path = _requirePathOwner(pathId);
_reconcileSupersededPendingReserve(pathId, path);
_reconcileInactiveOwnerBond(pathId, path);
+_expirePendingVersionIfPossible(pathId, path);
+if (path.pendingBundleHash == bytes32(0)) revert NoPendingVersion(pathId);
```

```
if (path.permanentlySlashed) revert SlashedPath(pathId);  
if (path.activeBundleHash != bytes32(0)) revert InitialProbationOnly(pathId);  
-if (path.pendingBundleHash == bytes32(0)) revert NoPendingVersion(pathId);
```

Status

Resolved



[L-06] PathBondManager#_reconcileSupersededPendingReserve - Cleared SupersededPending challenge keeps reserve locked while upheld releases it

Description

`_reconcileSupersededPendingReserve` releases the superseded pending reserve as soon as the bundle's `resolution` is `MaliciousBlocked`, but on a `Cleared` resolution it waits for `challengeableUntil` to elapse. This inverts the intuitive risk gradient: a bundle that was proven malicious unlocks the owner's funds immediately, while a bundle proven honest keeps them locked.

Vulnerability Details

```
function _reconcileSupersededPendingReserve(
    bytes32 pathId,
    PathTypes.PathCapital storage path
) private {
    bytes32 bundleHash = path.supersededPendingBundleHash;
    if (bundleHash == bytes32(0)) return;
    PathTypes.BundleRecord storage record = _bundleRecords[pathId][bundleHash];
    bool reserveStillLocked = record.challengeOpen
        || (record.resolution != PathTypes.BundleResolution.MaliciousBlocked
            && record.challengeableUntil != 0
            && record.challengeableUntil > block.timestamp);
    if (reserveStillLocked) return;
    uint256 reserve = path.ownerBondSupersededPending;
    path.supersededPendingBundleHash = bytes32(0);
    path.ownerBondSupersededPending = 0;
    ...
    path.ownerBondInactive += reserve;
}
```

src/PathBondManager.sol:1789-1808.

`_resolveClearedChallenge` simply closes the challenge and returns the bond to the challenger, leaving `record.resolution == Cleared` and `challengeableUntil` untouched:

```
function _resolveClearedChallenge(uint256 challengeId) private {
    ...
}
```

```

    _closeChallenge(
        challengeId, challenge, path, record,
        PathTypes.ChallengeStatus.Cleared,
        PathTypes.BundleResolution.Cleared
    );
    _transferAccountedPrompt(challenge.challenger, challenge.challengerBond);
    _syncPathAfterChallengeClose(challenge.pathId, path);
}

```

src/PathBondManager.sol:1344-1361.

The predicate `record.resolution != MaliciousBlocked` evaluates true for `Cleared`, so the reserve stays locked until `challengeableUntil`, even though the dispute is terminal.

Proof of Concept

```

function test_ClearedKeepsReserveLockedLongerThanMalicious() public {
    bytes32 pathA = _setupSupersededChallenge();
    bytes32 pathB = _setupSupersededChallenge();
    // resolve A as Cleared, B as Upheld
    vm.startPrank(resolver);
    manager.resolveChallenge(_chId(pathA), false, 0);
    manager.resolveChallenge(_chId(pathB), true, _maxSlash(pathB));
    vm.stopPrank();
    // touch reconcile via a topUp/bondInitial path
    _poke(pathA);
    _poke(pathB);
    // B (malicious) reserve unlocked immediately; A (cleared) still locked
    assertEq(manager.getPathCapital(pathB).ownerBondSupersededPending, 0);
    assertGt(manager.getPathCapital(pathA).ownerBondSupersededPending, 0);
}

```

Impact

An honest owner whose superseded pending bundle survives a frivolous challenge has their reserve locked longer than an owner whose bundle was found malicious. While the funds are eventually recoverable, the inverted incentive means honest behavior is punished with longer fund lockup, and the canonical reserve view stays stale until the timer elapses.

Recommendation

Treat `Cleared` as a terminal outcome that releases the reserve immediately, mirroring `MaliciousBlocked`:

```
-bool reserveStillLocked = record.challengeOpen
-  || (record.resolution != PathTypes.BundleResolution.MaliciousBlocked
-      && record.challengeableUntil != 0
-      && record.challengeableUntil > block.timestamp);
+bool resolutionTerminal = record.resolution ==
PathTypes.BundleResolution.MaliciousBlocked
+  || record.resolution == PathTypes.BundleResolution.Cleared;
+bool reserveStillLocked = record.challengeOpen
+  || (!resolutionTerminal
+      && record.challengeableUntil != 0
+      && record.challengeableUntil > block.timestamp);
```

Status

Resolved

[L-07] PathBondManager#_minimumUpheldSlash - Reads live minChallengeBond instead of challenge snapshot

Description

openChallenge snapshots minChallengeBond and challengerRewardBps into the challenge struct so resolution math is stable, but _minimumUpheldSlash reads the *live* minChallengeBond instead of the snapshot. An admin who calls setMinChallengeBond between open and resolve can shift the resolver's valid slash range mid-dispute.

Vulnerability Details

```
function setMinChallengeBond(
    uint256 newMinChallengeBond
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newMinChallengeBond == 0 || newMinChallengeBond > MAX_CHALLENGE_BOND) {
        revert InvalidAmount();
    }
    emit MinChallengeBondUpdated(minChallengeBond, newMinChallengeBond);
    minChallengeBond = newMinChallengeBond;
}
```

src/PathBondManager.sol:359-367.

openChallenge deliberately snapshots the values needed at resolution:

```
_challenges[challengeId] = PathTypes.Challenge({
    ...
    challengerBond: minChallengeBond,
    challengerRewardBpsSnapshot: challengerRewardBps,
    ...
});
```

src/PathBondManager.sol:948-958.

But the slash floor reads the live storage variable:

```
function _minimumUpheldSlash(
    PathTypes.Challenge storage challenge,
    uint256 maxSlash
) private view returns (uint256) {
```

```

    if (maxSlash == 0) revert InvalidAmount();
    uint256 minimumSlash = minChallengeBond;           // <- live, not snapshot
    uint256 rewardFloor = _minimumRewardSlash(challenge.challengerRewardBpsSnapshot);
    if (rewardFloor > minimumSlash) {
        minimumSlash = rewardFloor;
    }
    return minimumSlash > maxSlash ? maxSlash : minimumSlash;
}

```

src/PathBondManager.sol:1265-1277.

Proof of Concept

```

function test_LiveMinChallengeBondShiftsResolverRange() public {
    bytes32 pathId = _seedActiveBundle();
    uint256 challengeId = _openChallenge(pathId);
    uint256 maxSlash = manager.maxSlashAmount(pathId, ChallengeTarget.Active);
    uint256 originalMin = manager.minChallengeBond();
    uint256 inSnapshotRange = originalMin; // valid before parameter change
    // admin raises the floor mid-dispute
    vm.prank(admin);
    manager.setMinChallengeBond(originalMin * 10);
    // amount that was valid at open-time now reverts
    vm.prank(resolver);
    vm.expectRevert(PathBondManager.InvalidAmount.selector);
    manager.resolveChallenge(challengeId, true, inSnapshotRange);
}

```

Impact

Admin parameter changes after a challenge opens can change the resolver's allowed slash range. If `minChallengeBond` is raised, previously-valid resolution amounts revert; if it is lowered, the resolver can apply a smaller floor than the challenger reasonably expected when posting bond. Trust assumption: admin is benign, but the rest of the contract treats challenges as snapshot-stable, so this is an unintended live-read.

Recommendation

Snapshot `minChallengeBond` on the `Challenge` struct (analogous to `challengerBond` / `challengerRewardBpsSnapshot`) and use that snapshot in `_minimumUpheldSlash`:

```
_challenges[challengeId] = PathTypes.Challenge({  
    ...  
    challengerBond: minChallengeBond,  
+   minChallengeBondSnapshot: minChallengeBond,  
    challengerRewardBpsSnapshot: challengerRewardBps,  
    ...  
});
```

```
-uint256 minimumSlash = minChallengeBond;  
+uint256 minimumSlash = challenge.minChallengeBondSnapshot;
```

Status

Resolved

[L-08] PathBondManager#_resolveUpheldChallenge - Challenger reward can round to zero when bucket is below reward floor

Description

`_minimumUpheldSlash` computes a reward floor (the smallest slash that pays at least 1 wei of reward at the snapshot bps), but caps the floor at `maxSlash` when the slashable bucket is smaller than that floor. The payout then computes $(\text{appliedSlash} * \text{bps}) / \text{BPS_DENOMINATOR}$, which integer-divides to zero — the challenger receives only their bond back even on an upheld challenge.

Vulnerability Details

Floor calculation, capped at `maxSlash`:

```
function _minimumUpheldSlash(
    PathTypes.Challenge storage challenge,
    uint256 maxSlash
) private view returns (uint256) {
    if (maxSlash == 0) revert InvalidAmount();
    uint256 minimumSlash = minChallengeBond;
    uint256 rewardFloor = _minimumRewardSlash(challenge.challengerRewardBpsSnapshot);
    if (rewardFloor > minimumSlash) {
        minimumSlash = rewardFloor;
    }
    return minimumSlash > maxSlash ? maxSlash : minimumSlash;
}

function _minimumRewardSlash(
    uint16 challengerRewardBpsSnapshot
) private pure returns (uint256) {
    return (BPS_DENOMINATOR + challengerRewardBpsSnapshot - 1) /
    challengerRewardBpsSnapshot;
}
```

src/PathBondManager.sol:1265-1283.

Payout:

```
uint256 challengerReward =
    (appliedSlash * challenge.challengerRewardBpsSnapshot) / BPS_DENOMINATOR;
```

```

_transferAccountedPrompt(challenge.challenger, challenge.challengerBond +
challengerReward);
if (appliedSlash > challengerReward) {
    _transferAccountedPrompt(safetyFund, appliedSlash - challengerReward);
}

```

src/PathBondManager.sol:1417-1422.

If `maxSlash < rewardFloor`, the resolver can pass `slashAmount = maxSlash` (the only legal amount), and `(maxSlash * bps) / 10_000` rounds to zero. The challenger receives only their bond returned, despite a successful upheld outcome.

Proof of Concept

```

function test_ChallengerRewardRoundsToZero() public {
    // bps = 100 (1%), reward floor = ceil(10_000/100) = 100
    // craft a SupersededPending reserve of only 50 wei
    bytes32 pathId = _seedTinyReserveSuperseded(50);
    uint256 challengeId = _openSupersededChallenge(pathId);
    vm.prank(resolver);
    manager.resolveChallenge(challengeId, true, 50); // maxSlash == minSlash == 50
    // (50 * 100) / 10_000 = 0
    assertEq(_lastRewardPaid(challengeId), 0);
}

```

Impact

A challenger who successfully proves a malicious bundle but whose target bucket is small enough can earn zero reward. They risked frivolity penalties to defend the protocol but receive no incentive payment, weakening the economic security of small-bucket targets such as a dust `SupersededPending` reserve.

Recommendation

Round the reward up to at least 1 wei whenever any slash is applied, or pay any uncovered floor from the safety fund:

```

uint256 challengerReward =
    (appliedSlash * challenge.challengerRewardBpsSnapshot) / BPS_DENOMINATOR;

```

```
+if (appliedSlash > 0 && challengerReward == 0) {  
+    challengerReward = 1;  
+}
```

Alternatively, revert in `_minimumUpheldSlash` when the bucket is too small to fund a non-zero reward, and route those edge cases through the frivolous/cleared paths instead.

Status

Resolved

[L-09] PathBondManager#_recoverSigner - Accepts ECDSA-malleable signatures

Description

`_recoverSigner` uses `raw ecrecover` and accepts both `s` halves of the curve plus normalizes `v < 27` to `v + 27`, so each valid signature has multiple byte-distinct representations that recover the same signer. On-chain replay is prevented by the per-operation nonce, but off-chain systems that deduplicate authorizations by signature bytes can be bypassed.

Vulnerability Details

```
function _recoverSigner(
    bytes32 digest,
    bytes calldata signature
) private pure returns (address) {
    if (signature.length != 65) revert InvalidAuthorizationSigner(address(0));
    bytes32 r;
    bytes32 s;
    uint8 v;
    assembly {
        r := calldataload(signature.offset)
        s := calldataload(add(signature.offset, 0x20))
        v := byte(0, calldataload(add(signature.offset, 0x40)))
    }
    if (v < 27) v += 27;
    if (v != 27 && v != 28) revert InvalidAuthorizationSigner(address(0));
    address signer = ecrecover(digest, v, r, s);
    if (signer == address(0)) revert InvalidAuthorizationSigner(address(0));
    return signer;
}
```

src/PathBondManager.sol:1614-1634.

There is no `s <= secp256k1n/2` check (the `MALLEABILITY_THRESHOLD` enforced by OpenZeppelin's `ECDSA.tryRecover`). Given a valid `(r, s, v)` for some digest, an attacker can derive `(r, secp256k1n - s, v ^ 1)` (after the `v += 27` normalization) which recovers the same `authorizationSigner`.

Proof of Concept

```
function test_HighSAcceptedAndMatchesLowS() public {
    (uint8 v, bytes32 r, bytes32 s) = _signDigest(authSignerPk, digest);
    // canonical low-s recovers correctly
    assertEq(manager.exposedRecover(digest, abi.encodePacked(r, s, v)), authSigner);
    // flip s and v to produce malleable twin
    bytes32 sMal = bytes32(_n() - uint256(s));
    uint8 vMal = v ^ 1;
    assertEq(manager.exposedRecover(digest, abi.encodePacked(r, sMal, vMal)),
authSigner);
    // off-chain dedup keyed on signature bytes would treat these as distinct
}
```

Impact

On-chain, the per-operation nonce in `_consumeAuthorization` (`src/PathBondManager.sol:1530-1551`) prevents the malleable twin from being used for a second on-chain action. However, off-chain infrastructure (relayers, gating systems, signature-bytes-keyed dedup caches, monitoring) that hashes the raw 65-byte signature to detect duplicates can be tricked into treating the malleated copy as a fresh authorization, leading to double-processing or alert evasion.

Recommendation

```
if (v < 27) v += 27;
if (v != 27 && v != 28) revert InvalidAuthorizationSigner(address(0));
+if (uint256(s) > 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0)
{
+    revert InvalidAuthorizationSigner(address(0));
+}
address signer = ecrecover(digest, v, r, s);
```

Or replace the assembly recovery with OpenZeppelin's `ECDSA.tryRecover`, which already enforces canonical-`s` and rejects malleable signatures.

Status

Resolved



[L-10] PathBondManager#_consumeAuthorization - Signed `minAmount` has no upper bound, allowing over-bonding beyond signer intent

Description

The EIP-712 authorization signs `minAmount` and the on-chain check is only `amount >= minAmount`. There is no signed maximum, so an owner who holds a valid authorization can consume it with an `amount` arbitrarily larger than what the signer intended, depositing more bond than expected.

Vulnerability Details

`_consumeAuthorization` signs `minAmount` and only enforces a lower bound:

```
bytes32 structHash = keccak256(
    abi.encode(
        AUTHORIZATION_TYPEHASH,
        pathId,
        bundleHash,
        owner,
        riskTier,
        operation,
        minAmount,
        nonce,
        deadline
    )
);
```

`src/PathBondManager.sol:1535-1547.`

`_consumeReplacementAuthorization` does the same:

```
bytes32 structHash = keccak256(
    abi.encode(
        REPLACEMENT_AUTHORIZATION_TYPEHASH,
        pathId,
        previousBundleHash,
        bundleHash,
        owner,
        riskTier,
```

```

        minAmount,
        nonce,
        deadline
    )
};

```

src/PathBondManager.sol:1577-1588.

Earlier in the same helpers, the only amount-related guard is `if (amount < minAmount) revert AuthorizationAmountTooLow(amount, minAmount);`. There is no `amount <= maxAmount` check because no `maxAmount` is signed.

Proof of Concept

```

function test_OwnerOverBondsBeyondSignerIntent() public {
    bytes32 pathId = keccak256("path");
    bytes32 bundleHash = keccak256("bundle");
    uint256 signedMin = 1_000e18; // signer's intended size
    (bytes memory sig, uint256 nonce, uint64 deadline) = _signBondInitial(
        pathId, bundleHash, RiskTier.Low, signedMin, owner, signedMin
    );
    // owner consumes the same signature with 100x the size
    uint256 huge = 100_000e18;
    _fund(owner, huge);
    vm.prank(owner);
    manager.bondInitialAuthorized(
        pathId, bundleHash, RiskTier.Low, huge, owner, signedMin, nonce, deadline,
        sig
    );
    assertEq(manager.getPathCapital(pathId).ownerBondPending, huge);
}

```

Impact

The authorization signer cannot bound the maximum bond an owner deposits. If the signer intends to authorize "at least 1k PROMPT" for a particular path/bundle, the owner can consume the same signature for 100k PROMPT, locking funds beyond what was approved. While the owner is the only party who can pull tokens from themselves via `_pullExact`, this still violates the principle that the signer's policy is what's authoritative



— escrow magnitude is meaningful for risk-tier assignment, slashing exposure, and downstream off-chain accounting that may key on the signer's intended cap.

Recommendation

Sign a `maxAmount` (or the exact `amount`) and enforce `amount <= maxAmount`:

```
bytes32 structHash = keccak256(
    abi.encode(
        AUTHORIZATION_TYPEHASH,
        pathId,
        bundleHash,
        owner,
        riskTier,
        operation,
        minAmount,
+       maxAmount,
        nonce,
        deadline
    )
);
```

```
if (amount < minAmount) revert AuthorizationAmountTooLow(amount, minAmount);
+if (amount > maxAmount) revert AuthorizationAmountTooHigh(amount, maxAmount);
```

The corresponding EIP-712 typehash and any off-chain signing infrastructure must be updated accordingly.

Status

Acknowledged

[L-11] EpochRewardsDistributor#overrideEpochRoot - A single valid claim blocks cancelEpoch and overrideEpochRoot for the claim window

Description

overrideEpochRoot and cancelEpoch both refuse to run as soon as epoch.creatorClaimed != 0 || epoch.stakerClaimed != 0. Once a single legitimate claimant succeeds, the admin can no longer correct a faulty Merkle root nor cancel the epoch until the deadline passes. Recovery is reduced to waiting for sweepExpiredEpoch, which strands honest unclaimed users.

Vulnerability Details

_consumeClaim increments epoch.creatorClaimed or epoch.stakerClaimed on every successful claim (src/EpochRewardsDistributor.sol:333-345):

```
if (category == CATEGORY_CREATOR) {
    uint256 newClaimed = uint256(epoch.creatorClaimed) + amount;
    if (newClaimed > epoch.creatorPool) {
        revert PoolExceeded(category, newClaimed, epoch.creatorPool);
    }
    epoch.creatorClaimed = _toUint128(newClaimed);
} else {
    uint256 newClaimed = uint256(epoch.stakerClaimed) + amount;
    ...
    epoch.stakerClaimed = _toUint128(newClaimed);
}
```

Both admin recovery paths then guard on those counters being zero. From overrideEpochRoot (src/EpochRewardsDistributor.sol:175-177):

```
if (epoch.creatorClaimed != 0 || epoch.stakerClaimed != 0) {
    revert EpochClaimsAlreadyStarted(epochId);
}
```

And identically in cancelEpoch (src/EpochRewardsDistributor.sol:197-199):

```
if (epoch.creatorClaimed != 0 || epoch.stakerClaimed != 0) {
    revert EpochClaimsAlreadyStarted(epochId);
}
```

```
}
```

Because there is no per-leaf, per-account, or "no-bad-claim-yet" notion, **any** valid claim — even for 1 wei — flips the lock. After that the admin cannot patch a wrong leaf set with `overrideEpochRoot`, nor unwind a bad publish via `cancelEpoch`, until `block.timestamp > _effectiveClaimDeadline(epoch)`.

Proof of Concept

Step-by-step against the real contract:

1. Posting role calls `publishEpochRoot(epochId=1, merkleRoot=R, creatorPool=1_000e18, stakerPool=0, claimDeadline=now+30d)`. Pool is funded; `epoch.claimEligibleAt = now + 1h`.
2. After 1 hour, an honest creator who **is** in R calls `claim(1, CATEGORY_CREATOR, 1, proof)`. `_consumeClaim` succeeds, sets `epoch.creatorClaimed = 1`.
3. Off-chain analysis reveals the root R mistakenly omitted creator B who deserves 500e18.
4. Admin calls `overrideEpochRoot(1, R')` with the corrected root: reverts with `EpochClaimsAlreadyStarted(1)` because `epoch.creatorClaimed == 1 != 0`.
5. Admin instead calls `cancelEpoch(1, treasury)` to refund and republish: also reverts with `EpochClaimsAlreadyStarted(1)`.
6. Admin must wait until `block.timestamp > claimDeadline` (up to 30 days later) and call `sweepExpiredEpoch` to recover only `epoch.remaining`. Creator B never received their share for that epoch.

Impact

If an `EPOCH_POSTER_ROLE` holder publishes a Merkle root that omits or under-allocates legitimate users, a single early claimant from the published set permanently disables both correction tools for the entire window (up to `MAX_CLAIM_WINDOW = 90` days). Honest users who were left out of the bad root cannot be made whole until sweep, and even then the admin must rebuild and re-fund a fresh epoch from scratch — degrading reward UX and forcing manual off-chain compensation for the affected window.

Recommendation

Track whether a *malicious* / out-of-bounds claim has happened, or simply allow override/cancel but credit already-claimed amounts so honest claimants are protected:

```
function overrideEpochRoot(
    uint256 epochId,
    bytes32 newMerkleRoot
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newMerkleRoot == bytes32(0)) revert InvalidMerkleRoot();
    PathTypes.Epoch storage epoch = _epochs[epochId];
    if (!epoch.published) revert EpochNotPublished(epochId);
    uint64 deadline = _effectiveClaimDeadline(epoch);
    if (block.timestamp > deadline) {
        revert ClaimExpired(epochId, deadline);
    }
    - if (epoch.creatorClaimed != 0 || epoch.stakerClaimed != 0) {
    -     revert EpochClaimsAlreadyStarted(epochId);
    - }
    bytes32 previousRoot = epoch.merkleRoot;
    epoch.merkleRoot = newMerkleRoot;
    emit EpochRootOverridden(epochId, previousRoot, newMerkleRoot);
}
```

For `cancelEpoch`, refund only `epoch.remaining` (already correct) but drop the `creatorClaimed/stakerClaimed != 0` check; the existing claim-leaf de-dup via `_claimedLeaves` already prevents double-claims on the same leaf, but re-using an `epochId` after cancel must remain policy-controlled by the admin role. Alternatively, introduce a pause-and-revert path that requires the admin to also pause first, mark the epoch superseded, and let users migrate via a fresh epoch.

Status

Resolved

[L-12] EpochRewardsDistributor#overrideEpochRoot - Does not refresh claimEligibleAt

Description

publishEpochRoot sets epoch.claimEligibleAt = now + CLAIM_DELAY to give users a one-hour observation window before a fresh root becomes claimable. overrideEpochRoot swaps in a brand new root but never updates claimEligibleAt, so a corrected root posted after the original delay has elapsed becomes immediately claimable with zero observation period.

Vulnerability Details

On publish, claimEligibleAt is initialised with the 1-hour buffer (src/EpochRewardsDistributor.sol:152):

```
epoch.claimEligibleAt = _toUint64(block.timestamp + CLAIM_DELAY);
```

Override then mutates only merkleRoot (src/EpochRewardsDistributor.sol:179-182):

```
bytes32 previousRoot = epoch.merkleRoot;
epoch.merkleRoot = newMerkleRoot;
emit EpochRootOverridden(epochId, previousRoot, newMerkleRoot);
```

_consumeClaim gates only on the original claimEligibleAt (src/EpochRewardsDistributor.sol:316-320):

```
PathTypes.Epoch storage epoch = _epochs[epochId];
if (!epoch.published) revert EpochNotPublished(epochId);
if (block.timestamp < epoch.claimEligibleAt) {
    revert ClaimNotYetEligible(epochId, epoch.claimEligibleAt);
}
```

Because claimEligibleAt snapshots the *original publish* time + CLAIM_DELAY, replacing the root any time after publish + 1 hour makes the new root claimable in the same block as the override.

Proof of Concept

Step-by-step against the real contract:

1. $t = T_0$: poster calls publishEpochRoot(epochId=1, merkleRoot=R,

#hashlock.

`creatorPool=1_000e18, stakerPool=0, claimDeadline=T0 + 7d`). Storage:
`epoch.claimEligibleAt = T0 + 3600`.

2. `t = T0 + 1d`: admin (or a compromised admin key) calls `overrideEpochRoot(1, R')` where `R'` includes a leaf `(1, attacker, CATEGORY_CREATOR, 1_000e18)`. The function only writes `epoch.merkleRoot = R'`; `claimEligibleAt` is still `T0 + 3600`, which is now in the past.

3. ****Same block****, attacker calls `claim(1, CATEGORY_CREATOR, 1_000e18, proof)`. The `block.timestamp < epoch.claimEligibleAt` check passes (`T0 + 1d > T0 + 3600`), the proof verifies against `R'`, and the full pool is paid out. No off-chain monitor had any opportunity to react.

Impact

The 1-hour delay exists so users (and monitoring infrastructure) can detect and dispute a fraudulent or buggy root before any value moves. When the admin replaces a root via `overrideEpochRoot`, the new leaf set bypasses that window entirely if the original delay has lapsed. A compromised or mistaken admin can swap in a self-favoring root and drain the epoch within one block — eliminating any reaction window for guardians/users to pause claims or alert the team.

Recommendation

Reset the claim eligibility window when the root is overridden:

```
function overrideEpochRoot(
    uint256 epochId,
    bytes32 newMerkleRoot
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newMerkleRoot == bytes32(0)) revert InvalidMerkleRoot();
    PathTypes.Epoch storage epoch = _epochs[epochId];
    if (!epoch.published) revert EpochNotPublished(epochId);
    uint64 deadline = _effectiveClaimDeadline(epoch);
    if (block.timestamp > deadline) {
        revert ClaimExpired(epochId, deadline);
    }
    if (epoch.creatorClaimed != 0 || epoch.stakerClaimed != 0) {
        revert EpochClaimsAlreadyStarted(epochId);
    }
}
```

```
    }  
    bytes32 previousRoot = epoch.merkleRoot;  
    epoch.merkleRoot = newMerkleRoot;  
+   uint64 newEligibleAt = _toUint64(block.timestamp + CLAIM_DELAY);  
+   if (newEligibleAt > epoch.claimEligibleAt) {  
+       epoch.claimEligibleAt = newEligibleAt;  
+   }  
    emit EpochRootOverridden(epochId, previousRoot, newMerkleRoot);  
}
```

Also extend `claimDeadline` (or require the admin to pass a new one) so the override does not silently shorten the effective claim window after the delay reset.

Status

Resolved

[L-13] EpochRewardsDistributor#unpause - Bypasses admin/pauser separation

Description

The constructor and `_validatePrivilegedRoleAssignment` enforce that `DEFAULT_ADMIN_ROLE`, `EPOCH_POSTER_ROLE`, and `PAUSER_ROLE` are held by three distinct accounts. `unpause()` violates this invariant by accepting either `PAUSER_ROLE` or `DEFAULT_ADMIN_ROLE`, granting the admin half of the pauser's authority despite the contract explicitly forbidding role overlap.

Vulnerability Details

The constructor and `grant validator` enforce strict separation (`src/EpochRewardsDistributor.sol:69-71` and `386-412`):

```
if (admin == epochPoster || admin == pauser || epochPoster == pauser) {
    revert DistinctRoleHoldersRequired();
}
```

```
} else if (role == PAUSER_ROLE) {
    if (hasRole(DEFAULT_ADMIN_ROLE, account)) {
        revert PrivilegedRoleConflict(account, role, DEFAULT_ADMIN_ROLE);
    }
    if (hasRole(EPOCH_POSTER_ROLE, account)) {
        revert PrivilegedRoleConflict(account, role, EPOCH_POSTER_ROLE);
    }
}
```

`pause()` correctly limits authority to `PAUSER_ROLE` (`src/EpochRewardsDistributor.sol:88-91`):

```
function pause() external onlyRole(PAUSER_ROLE) {
    _pausedAt = _toUint64(block.timestamp);
    _pause();
}
```

But `unpause()` opens the gate to `DEFAULT_ADMIN_ROLE` as well (`src/EpochRewardsDistributor.sol:93-102`):


```
function unpause() external {
    if (!hasRole(PAUSER_ROLE, msg.sender) && !hasRole(DEFAULT_ADMIN_ROLE,
msg.sender)) {
        revert AccessControlUnauthorizedAccount(msg.sender, PAUSER_ROLE);
    }
    if (_pausedAt != 0) {
        totalPausedDuration += _toUint64(block.timestamp) - _pausedAt;
        _pausedAt = 0;
    }
    _unpause();
}
```

The invariant the rest of the contract advertises — that the admin key cannot unilaterally control the pause switch — is silently broken on unpause.

Proof of Concept

```
function test_AdminUnilaterallyUnpausesAfterPauserAction() public {
    // Pauser correctly halts the contract during an incident.
    vm.prank(pauser);
    distributor.pause();
    assertTrue(distributor.paused());
    // Admin alone reverses it without the pauser's consent.
    vm.prank(admin);
    distributor.unpause();
    assertFalse(distributor.paused());
    // The two-party safety invariant is broken: a compromised admin key
    // can keep claims live throughout an attempted pause.
}
```

Impact

A compromised admin key alone can resume claims without the pauser's consent, defeating the purpose of separating the roles in the first place. An incident-response pause initiated by the pauser to halt suspicious claims can be reverted by the admin in one transaction, allowing attacks (e.g., an exploited or fraudulent epoch root) to continue to drain the pool. This collapses the two-party "circuit breaker" guarantee the constructor/grant logic enforces elsewhere.

Recommendation

Restrict `unpause()` to the same role that can `pause()`, matching the documented separation-of-duties model:

```
- function unpause() external {  
-     if (!hasRole(PAUSER_ROLE, msg.sender) && !hasRole(DEFAULT_ADMIN_ROLE,  
msg.sender)) {  
-         revert AccessControlUnauthorizedAccount(msg.sender, PAUSER_ROLE);  
-     }  
+ function unpause() external onlyRole(PAUSER_ROLE) {  
    if (_pausedAt != 0) {  
        totalPausedDuration += _toUint64(block.timestamp) - _pausedAt;  
        _pausedAt = 0;  
    }  
    _unpause();  
}
```

If the team genuinely wants admin to be able to override a stuck pauser, document that explicitly and remove the equally-strict `DistinctRoleHoldersRequired` / `PrivilegedRoleConflict` invariants — the two designs are mutually inconsistent today.

Status

Acknowledged

[L-14] MappingRewardsDistributor#setClaimableBatch - SET-batch front-run can overpay a user

Description

`setClaimableBatch` writes **absolute** balances rather than deltas. A user who already has unclaimed `oldAmount` and is about to be rewritten to a higher `newAmount` can front-run the funder's batch transaction by claiming `oldAmount` first; the batch then resets their bucket to the full `newAmount` against a freshly-zeroed slot, so the user effectively receives `oldAmount + newAmount` instead of `newAmount`.

Vulnerability Details

`setClaimableBatch` invokes `_setClaimableAmount` per entry (src/MappingRewardsDistributor.sol:73-93):

```
function setClaimableBatch(
    address[] calldata accounts,
    uint8[] calldata categories,
    uint256[] calldata amounts,
    uint256 batchId
) external onlyRole(FUNDER_ROLE) {
    if (accounts.length == 0) revert InvalidAmount();
    if (accounts.length != categories.length || accounts.length != amounts.length) {
        revert MismatchedArrayLengths();
    }
    if (usedBatchIds[batchId]) revert BatchAlreadyApplied(batchId);
    usedBatchIds[batchId] = true;
    uint256 count = accounts.length;
    for (uint256 i = 0; i < count; ++i) {
        _setClaimableAmount(accounts[i], categories[i], amounts[i], batchId);
    }
    ...
}
```

`_setClaimableAmount` does an absolute write (src/MappingRewardsDistributor.sol:247-277):

```
uint256 oldAmount = _claimable[account][category];
```

```

_claimable[account][category] = amount;
if (amount > oldAmount) {
    uint256 delta = amount - oldAmount;
    totalClaimable += delta;
    if (category == CATEGORY_CREATOR) {
        creatorClaimableTotal += delta;
    } else {
        stakerClaimableTotal += delta;
    }
} else if (amount < oldAmount) {
    uint256 delta = oldAmount - amount;
    totalClaimable -= delta;
    ...
}

```

`claim` simply decrements the same slot (`src/MappingRewardsDistributor.sol:226-245`):

```

function _consume(
    address account,
    uint8 category,
    uint256 amount
) internal returns (uint256) {
    _requireValidCategory(category);
    if (amount == 0) revert InvalidAmount();
    uint256 currentAmount = _claimable[account][category];
    if (currentAmount < amount) revert NotEnoughClaimableTokens();
    _claimable[account][category] = currentAmount - amount;
    totalClaimable -= amount;
    ...
}

```

Because the funder's transaction is publicly visible in the mempool with the new absolute amounts, a target user can sandwich it: claim their existing `oldAmount` first, then let the batch overwrite their slot to `newAmount`. There is no per-account nonce and no "expected previous amount" check.

Proof of Concept

```
function test_FrontRunSetBatchOverpays() public {
    // Setup: funder gave Alice 100e18 in a previous batch; contract is enabled and
    solvent.
    address alice = makeAddr("alice");
    uint256 oldAmount = 100e18;
    uint256 newAmount = 250e18;
    vm.prank(funder);
    address[] memory a = new address[](1);
    uint8[] memory c = new uint8[](1);
    uint256[] memory v = new uint256[](1);
    a[0] = alice; c[0] = 0; v[0] = oldAmount;
    distributor.setClaimableBatch(a, c, v, 1);
    vm.prank(admin);
    distributor.setDisabled(false);
    // Funder broadcasts a tx setting Alice -> newAmount (batchId=2). Alice sees it
    in mempool.
    // Alice front-runs:
    vm.prank(alice);
    uint256 claimed1 = distributor.claim(0, oldAmount); // succeeds, slot = 0
    assertEq(claimed1, oldAmount);
    assertEq(distributor.claimable(alice, 0), 0);
    // Funder's batch lands second: slot is overwritten to newAmount (delta =
    newAmount - 0).
    vm.prank(funder);
    distributor.setClaimableBatch(a, c, _u256(newAmount), 2);
    assertEq(distributor.claimable(alice, 0), newAmount);
    // Alice claims again:
    vm.prank(alice);
    uint256 claimed2 = distributor.claim(0, newAmount);
    assertEq(claimed2, newAmount);
    // Total received: oldAmount + newAmount, not newAmount.
    assertEq(claimed1 + claimed2, oldAmount + newAmount);
}

function _u256(uint256 x) internal pure returns (uint256[] memory r) {
    r = new uint256[](1); r[0] = x;
}
```

Impact

Any user whose claimable bucket is being increased by a SET batch (e.g., `oldAmount = 100e18` rising to `newAmount = 250e18`) can capture an extra `oldAmount` worth of PROMPT by front-running. The overpayment is bounded by the user's pre-batch `_claimable[account][category]`, so a single batch caps loss per victim at their previous unclaimed balance — but across a large batch, multiple recipients can do this simultaneously, draining `totalClaimable` faster than the funder accounted for. This silently inflates protocol distribution and can drive the contract toward `DistributorInsolvent`.

Recommendation

Either switch the batch API to deltas, or pass an `expectedOldAmount` per entry and revert on mismatch:

```
function setClaimableBatch(
    address[] calldata accounts,
    uint8[] calldata categories,
-   uint256[] calldata amounts,
+   uint256[] calldata expectedOldAmounts,
+   uint256[] calldata newAmounts,
    uint256 batchId
) external onlyRole(FUNDER_ROLE) {
    if (accounts.length == 0) revert InvalidAmount();
-   if (accounts.length != categories.length || accounts.length != amounts.length) {
+   if (
+       accounts.length != categories.length
+       || accounts.length != newAmounts.length
+       || accounts.length != expectedOldAmounts.length
+   ) {
        revert MismatchedArrayLengths();
    }
    if (usedBatchIds[batchId]) revert BatchAlreadyApplied(batchId);
    usedBatchIds[batchId] = true;
    uint256 count = accounts.length;
    for (uint256 i = 0; i < count; ++i) {
+       if (_claimable[accounts[i]][categories[i]] != expectedOldAmounts[i]) {
```

```
+         revert StaleBatch();  
+     }  
-     _setClaimableAmount(accounts[i], categories[i], amounts[i], batchId);  
+     _setClaimableAmount(accounts[i], categories[i], newAmounts[i], batchId);  
    }  
    ...  
}
```

Alternatively, expose only an `addClaimable` (delta-add) primitive so absolute writes are not racy. If absolute semantics must be preserved, route batches through a private mempool / commit-reveal so they cannot be front-run.

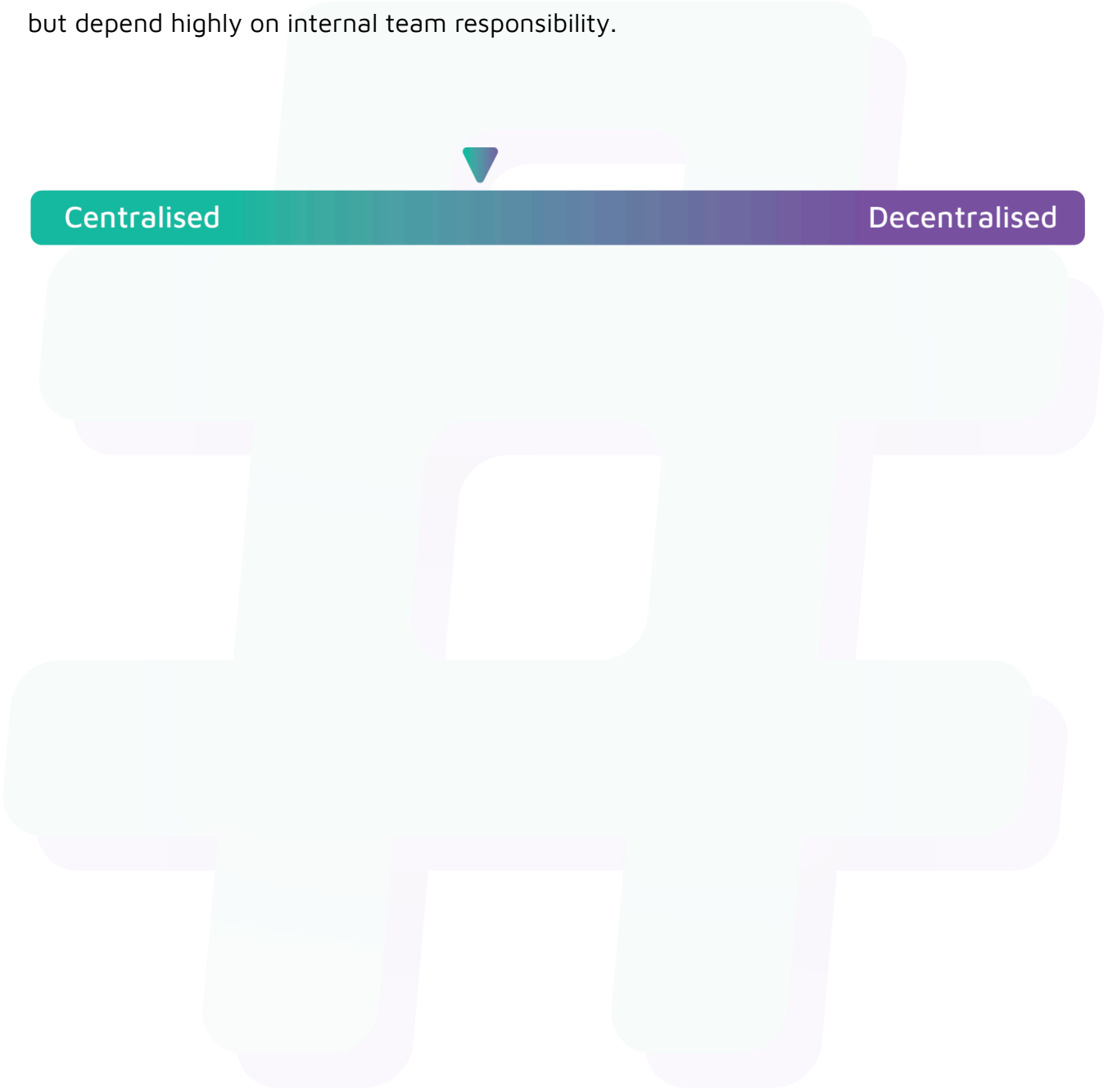
Status

Resolved

Centralisation

The Wayfinder project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Wayfinder project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with Resolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.